

Implementação e Validação de um Controlador de Cache Direct-Mapped para Arquitetura RISC-V

Felipe Barros Ratton de Almeida¹, Laura Menezes Heráclito Alves¹, Mateus Ribeiro Fernandes¹, and Vitor de Meira Gomes¹

¹ICEI — Instituto de Ciências Exatas e Informática, PUC Minas, Brasil

<https://github.com/nkdwon/ac3-tp01-riscv-cache-controller>

Resumo

Este trabalho apresenta a implementação e validação de um controlador de cache *direct-mapped* em SystemVerilog para uma arquitetura baseada em RISC-V, seguindo a especificação do Capítulo 5 de Patterson e Hennessy [1]. O projeto contempla operações de leitura e escrita com detecção de *cache hits* e *cache misses*, gerenciamento dos bits *valid* e *dirty*, política de escrita *write-back* e estratégia de alocação *write-allocate*. Além do controlador, foram desenvolvidos modelos simplificados de CPU e memória principal para compor o ambiente de simulação. A validação funcional foi realizada por meio de testes automatizados em Icarus Verilog [2], cobrindo leitura, escrita, substituição de linhas, consistência de dados e casos limite, com análise de formas de onda no GTKWave [3]. Os resultados confirmam o correto funcionamento do controlador em todos os cenários avaliados.

Palavras-chave: RISC-V; Cache; Direct-Mapped Cache; Write-Back; Computer Architecture.

1 Introdução

O desempenho dos sistemas computacionais modernos é fortemente limitado pela diferença de velocidade entre processador e memória principal — fenômeno conhecido como *Memory Wall*. Enquanto a capacidade de processamento evoluiu de forma expressiva nas últimas décadas, a latência de acesso à memória DRAM não acompanhou esse crescimento na mesma proporção, criando um gargalo que compromete o desempenho geral do sistema.

A principal solução adotada pela indústria é a utilização de memórias cache: camadas intermediárias de alta velocidade posicionadas entre o processador e a memória principal. O funcionamento eficiente das caches baseia-se em dois princípios observados na execução de programas reais. A **localidade temporal** indica que dados recentemente acessados tendem a ser acessados novamente em breve. A **localidade espacial** indica que, ao acessar uma posição de memória, posições próximas também serão provavelmente acessadas em seguida. Esses princípios justificam o carregamento de blocos inteiros na cache a cada acesso.

Entre as organizações de cache possíveis — mapeamento direto, associativo por conjuntos e totalmente associativo — a cache *direct-mapped* destaca-se pela simplicidade de implementação e baixo custo de hardware. Nesse modelo, cada bloco da memória principal pode ocupar exatamente uma linha da cache, determinada diretamente por parte dos bits do endereço.

Este trabalho tem como objetivo implementar e validar um controlador de cache *direct-mapped* em SystemVerilog, incorporando política de escrita *write-back* e estratégia de alocação *write-allocate*, conforme a especificação do Capítulo 5 de [1]. Além do controlador, foram desenvolvidos modelos de CPU e memória principal para compor um ambiente completo de simulação. A verificação funcional foi realizada por testes automatizados que cobrem os principais cenários operacionais, incluindo operações de leitura e escrita, substituição de linhas, conflitos de mapeamento, consistência dos dados e situações limite.

2 Fundamentação Teórica

2.1 Organização da Cache Direct-Mapped

Na cache *direct-mapped*, cada bloco da memória principal possui uma única posição possível dentro da cache, determinada por um subconjunto dos bits do endereço. Essa característica simplifica o hardware de busca, mas pode gerar conflitos quando múltiplos blocos competem pela mesma linha.

Na implementação desenvolvida, endereços de 32 bits são divididos em três campos:

$$\begin{array}{ccc} \underbrace{\text{Tag}}_{18 \text{ bits}} & \underbrace{\text{Índice}}_{10 \text{ bits}} & \underbrace{\text{Offset}}_{4 \text{ bits}} \end{array}$$

Os 10 bits de índice resultam em $2^{10} = 1024$ linhas de cache. O campo *offset* de 4 bits endereça posições dentro do bloco. A *tag* de 18 bits identifica qual bloco da memória principal está armazenado em cada linha.

Cada linha da cache armazena, além dos dados, três informações de controle:

- **Tag:** identifica o bloco armazenado naquela linha;
- **Valid bit:** indica se a linha contém dados válidos. Após o reset, todas as linhas são marcadas como inválidas;
- **Dirty bit:** indica se o conteúdo da linha foi modificado após o carregamento da memória principal.

2.2 Cache Hit e Cache Miss

Ao receber uma requisição da CPU, o controlador verifica se o bloco solicitado já está presente na cache. Um **cache hit** ocorre quando o *valid bit* da linha está ativo e a tag armazenada coincide com a tag do endereço solicitado. Nesse caso, o dado é fornecido diretamente pela cache, sem acesso à memória principal.

Um **cache miss** ocorre quando o bloco não está presente ou a linha é inválida. Nesse caso, o controlador precisa carregar o bloco da memória principal antes de atender a requisição,

incorrendo em penalidade de tempo.

2.3 Política de Escrita: Write-Back

Na política *write-back*, operações de escrita atualizam apenas a cache, sem propagar imediatamente as modificações para a memória principal. A linha modificada é marcada através do *dirty bit*. A atualização da memória principal ocorre somente quando o bloco precisa ser substituído por outro — nesse momento, o controlador realiza um *write-back*, copiando o conteúdo modificado para a memória antes de sobrescrever a linha. Essa abordagem reduz significativamente o tráfego de escrita na memória principal.

2.4 Estratégia de Alocação: Write-Allocate

Em caso de *write miss* — tentativa de escrita em um bloco ausente da cache — a política *write-allocate* determina que o bloco deve ser primeiro carregado da memória principal para a cache, e somente então a escrita é realizada. Essa estratégia é naturalmente combinada com *write-back* e tende a apresentar melhor desempenho quando há reutilização futura do bloco recém-escrito.

2.5 Máquina de Estados Finitos (FSM)

O comportamento do controlador foi modelado por meio de uma Máquina de Estados Finitos (*Finite State Machine* — FSM). Essa abordagem organiza claramente o fluxo de execução, tornando o sistema previsível e facilitando a validação. A FSM é responsável por: receber e decodificar requisições da CPU; detectar *hits* e *misses*; coordenar leituras e *write-backs* com a memória principal; atualizar os bits de controle; e retornar dados ao processador.

3 Implementação

3.1 Estrutura do Projeto

O repositório foi organizado de forma modular para facilitar o desenvolvimento e a manutenção:

- `src/` — módulos principais da cache (`dm_cache_data.sv`, `dm_cache_tag.sv`, `dm_cache_fsm.sv`);
- `tb/` — ambiente de testes automatizados e testbench principal;
- `debug/` — testbenches auxiliares utilizados durante o desenvolvimento;
- `docs/` — documentação e relatório do projeto;
- `bin/` — artefatos gerados durante a simulação.

3.2 Módulos Principais

`dm_cache_data.sv`: responsável pelo armazenamento efetivo dos dados das 1024 linhas da cache. Fornece mecanismos de leitura e escrita de blocos. Em operações de *hit*, os dados são obtidos diretamente deste módulo sem acesso à memória principal.

dm_cache_tag.sv: armazena as informações de controle de cada linha — *tag*, *valid bit* e *dirty bit*. Essas informações são consultadas pela FSM a cada acesso para determinar se ocorreu *hit* ou *miss* e se é necessário realizar *write-back* antes de uma substituição.

dm_cache_fsm.sv: núcleo do controlador. Implementa a FSM que coordena todas as operações da cache, gerencia a comunicação com CPU e memória principal, e atualiza os bits de controle conforme cada cenário.

memory_model.sv: modelo simplificado da memória principal utilizado no ambiente de simulação. Responde a requisições de leitura e escrita geradas pela FSM, permitindo validar corretamente operações de carregamento de blocos e *write-back*.

cpu_request_model.sv: modelo de CPU que gera requisições de leitura e escrita para a cache, aguardando o sinal *cpu_ready* antes de emitir a próxima operação. Sua utilização permitiu automatizar completamente os cenários de teste.

3.3 Tratamento de Cache Hit

Quando a CPU realiza uma requisição, o controlador consulta o módulo de tags. Se o *valid bit* estiver ativo e a tag coincidir, ocorre *hit*:

- **Leitura**: o dado é obtido diretamente da cache e retornado à CPU no mesmo ciclo, sem acesso à memória principal;
- **Escrita**: o dado é atualizado na linha local, o *dirty bit* é ativado, e nenhum acesso à memória é realizado.

3.4 Tratamento de Cache Miss

Em caso de *miss*, a FSM executa o seguinte procedimento:

1. Verifica o estado da linha a ser substituída;
2. Se o *dirty bit* estiver ativo, realiza *write-back* — copia o conteúdo modificado para a memória principal;
3. Solicita o novo bloco à memória principal (*write-allocate* em caso de escrita);
4. Atualiza a tag e ativa o *valid bit*;
5. Executa a operação solicitada (leitura ou escrita) e retorna o dado à CPU.

Esse procedimento garante a consistência entre cache e memória principal em qualquer situação.

4 Metodologia de Testes

A validação funcional foi realizada por meio do testbench principal **tb_dm_cache.sv**, responsável por gerar clock e reset, instanciar os modelos de CPU, cache e memória, e verificar automaticamente os resultados de cada operação através da tarefa **verifica()**, que compara os valores observados com os esperados. Ao final da simulação, o contador global de erros é analisado; a ausência de falhas resulta na mensagem **SIMULACAO FINALIZADA COM SUCESSO**.

O ambiente monitora continuamente: leituras e escritas realizadas na memória principal; atualizações dos bits *valid* e *dirty*; alterações de tags; e operações de substituição e *write-back*. Essa abordagem permite verificar não apenas os dados retornados à CPU, mas também o comportamento interno da cache.

Os cenários de teste foram divididos em cinco grupos, conforme descrito nas subseções a seguir.

4.1 Testes de Leitura

Implementados em `tests_read.sv`. O primeiro acesso a um endereço não presente na cache deve resultar em *miss*, com leitura da memória principal, atualização do *valid bit* e armazenamento correto da tag. O acesso subsequente ao mesmo endereço deve resultar em *hit*, sem novo acesso à memória.

4.2 Testes de Escrita

Implementados em `tests_write.sv`. Verificam *write hit* (atualização da linha com ativação do *dirty bit* e sem acesso à memória) e *write miss* (comportamento *write-allocate*: carregamento do bloco seguido da escrita e ativação do *dirty bit*). Múltiplas escritas consecutivas na mesma linha também foram avaliadas para confirmar a preservação do *dirty bit*.

4.3 Testes de Substituição

Implementados em `tests_replacement.sv`. A cache foi completamente preenchida com 1024 linhas, e em seguida foram forçados conflitos de mapeamento. Dois casos foram avaliados: substituição de linha limpa (sem *write-back*) e substituição de linha *dirty* (com *write-back* obrigatório antes do carregamento do novo bloco).

4.4 Testes de Consistência

Implementados em `tests_consistency.sv`. Verificam a preservação dos dados após múltiplas operações. Valores distintos foram gravados em diferentes linhas e relidos sucessivamente para confirmar a ausência de corrupção. Também foram realizados acessos alternados a dois endereços mapeados para o mesmo índice, validando a consistência após substituições repetidas.

4.5 Testes de Casos Limite

Implementados em `tests_edge_cases.sv`. Verificam o comportamento do sistema nos extremos do espaço de endereçamento — menor e maior endereço alinhado — confirmando o cálculo correto dos campos de índice e tag e o funcionamento adequado do *valid bit* nessas posições.

A Tabela 1 resume todos os cenários e seus resultados.

Tabela 1: Resumo dos testes realizados e resultados obtidos.

Cenário	Comportamento Esperado	Resultado
Estado inicial após reset	valid/dirty = 0 em todas as linhas	PASS
Read miss (1ª leitura)	acesso à memória + atualiza tag	PASS
Read hit (leitura repetida)	sem acesso à memória	PASS
Write hit	dirty = 1, sem acesso à memória	PASS
Write miss (write-allocate)	carrega bloco + escreve + dirty	PASS
Substituição de linha limpa	sem write-back	PASS
Substituição de linha dirty	write-back executado	PASS
Consistência após conflitos	dados preservados	PASS
Endereços extremos (min/max)	funcionamento normal	PASS

5 Resultados e Análise

Todos os testes foram executados com sucesso pelo simulador Icarus Verilog, resultando na mensagem `SIMULACAO FINALIZADA COM SUCESSO`. As formas de onda geradas no GTKWave permitiram inspecionar o comportamento interno do controlador em cada cenário, complementando as verificações automáticas do testbench.

5.1 Estado Inicial e Reset

A Figura 1 apresenta os sinais internos imediatamente após a aplicação do reset. O estado da FSM (`rstate[1:0]`) permanece em 00 (IDLE) e os sinais `tag_we` e `tag_write_valid` ficam em zero durante todo o período, confirmando que nenhuma linha é inicializada com dados válidos. Isso garante que os primeiros acessos após o reset resultem inevitavelmente em *cache miss*.

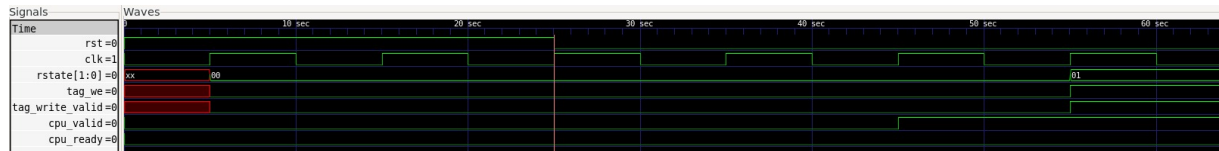


Figura 1: Sinais internos após reset: FSM em estado IDLE, `tag_we` e `tag_write_valid` em zero, confirmando inicialização correta de todas as linhas como inválidas.

5.2 Operações de Leitura

A Figura 2 apresenta os testes de leitura. No primeiro acesso a cada endereço, o sinal `mem_valid` sobe, indicando requisição à memória principal (*read miss*). Após o carregamento do bloco, `cpu_ready` sobe confirmando a entrega do dado à CPU. Nos acessos subsequentes ao mesmo endereço, `mem_valid` permanece em zero — caracterizando *read hit* e demonstrando que o bloco foi corretamente armazenado na cache.

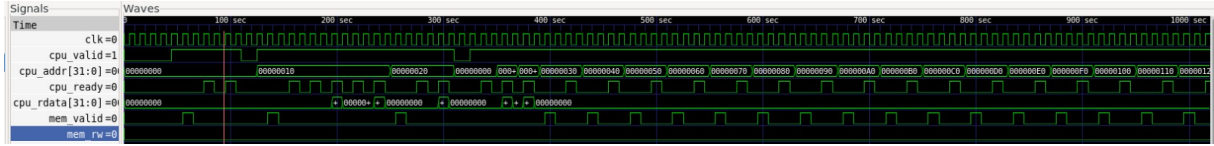


Figura 2: Testes de leitura: *mem_valid* alto apenas no primeiro acesso (*read miss*); acessos subsequentes ao mesmo endereço são atendidos diretamente pela cache (*read hit*).

A Figura 3 detalha o fluxo de um *read miss* com alocação, mostrando os sinais *mem_addr* e *mem_rdata* durante o carregamento do bloco e a subsequente atualização da linha de cache.

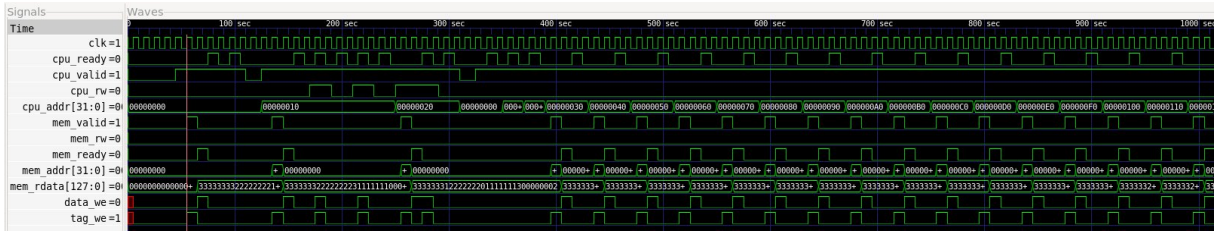


Figura 3: Fluxo completo de *read miss* com alocação: *mem_addr* aponta o bloco solicitado, *mem_rdata* transporta o conteúdo carregado e *tag_we* sobe confirmando a atualização da linha.

5.3 Operações de Escrita

A Figura 4 apresenta os testes de escrita. Em *write hit*, o sinal *tag_write_dirty* sobe e *mem_valid* permanece baixo, confirmando que apenas a cache foi atualizada — comportamento correto da política *write-back*. Em *write miss*, o controlador primeiro executa a alocação do bloco (conforme *write-allocate*) e somente então realiza a escrita, ativando o *dirty bit*.

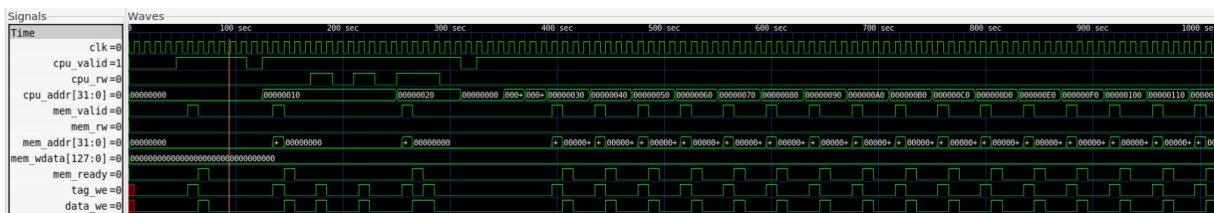


Figura 4: Testes de escrita: em *write hit*, *tag_write_dirty* sobe sem acesso à memória; em *write miss*, o bloco é alocado antes da escrita (*write-allocate*).

5.4 Substituição com Write-Back

As Figuras 5 e 6 ilustram o cenário de substituição de linha com *dirty bit* ativo — o caso mais crítico do projeto. O sinal *mem_rw* sobe indicando escrita na memória principal (*write-back*), e *mem_wdata* transporta o conteúdo modificado da linha. Somente após a conclusão do *write-back* o novo bloco é carregado. Após a substituição, o *dirty bit* é zerado e a nova tag é armazenada, restaurando a consistência entre cache e memória principal.

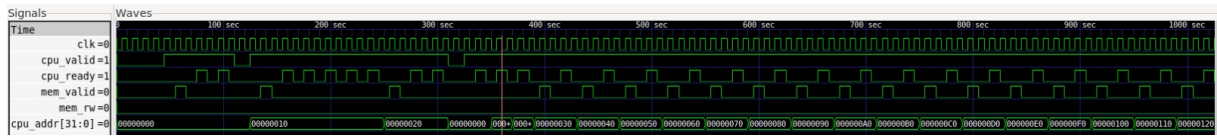


Figura 5: Substituição com *write-back*: *mem_rw* alto indica a propagação do bloco modificado para a memória principal antes do carregamento do novo bloco.



Figura 6: Detalhe do *write-back* com *dirty bit* ativo: *mem_wdata* carrega o conteúdo da linha antes da substituição, garantindo a integridade dos dados na memória principal.

5.5 Discussão Geral

Os resultados obtidos demonstram que o controlador implementado atende integralmente a todos os requisitos funcionais especificados. A política *write-back* mostrou-se eficiente ao concentrar as escritas na memória principal apenas nos momentos estritamente necessários — substituições de linhas *dirty* — reduzindo o tráfego de memória em comparação com a alternativa *write-through*. A política *write-allocate* garantiu o correto aproveitamento dos blocos recém-escritos, favorecendo acessos futuros ao mesmo conjunto de dados.

Nos testes de consistência, nenhuma corrupção de dados foi observada mesmo após conflitos sucessivos de índice com múltiplas substituições. Nos casos limite, os campos de índice e tag foram calculados corretamente para os endereços extremos do espaço de endereçamento, confirmando a ausência de falhas de contorno.

6 Conclusão

O controlador de cache *direct-mapped* implementado em SystemVerilog atende corretamente a todos os requisitos funcionais do trabalho. A abordagem modular — separando armazenamento de dados (*dm_cache_data*), informações de controle (*dm_cache_tag*) e lógica de controle (*dm_cache_fsm*) — facilitou tanto o desenvolvimento quanto o processo de depuração e validação.

Os testes automatizados desenvolvidos cobriram todos os cenários obrigatórios: operações de leitura e escrita com *hit* e *miss*, substituição de linhas limpas e *dirty*, *write-back*, *write-allocate*, consistência dos dados após conflitos de índice e funcionamento nos casos limite. A análise das formas de onda no GTKWave confirmou o comportamento esperado em cada cenário, evidenciando a correta transição entre os estados da FSM e a integridade dos dados ao longo de toda a simulação.

Além dos aspectos técnicos, o projeto proporcionou experiência prática com linguagens de descrição de hardware, ambientes de simulação digital e metodologias de verificação

funcional baseadas em testes automatizados, consolidando os conceitos de hierarquia de memória estudados na disciplina de Arquitetura de Computadores.

Dificuldades Encontradas

A principal dificuldade do projeto foi garantir a sincronização correta entre a FSM e os modelos de CPU e memória, especialmente nos ciclos em que um *write-back* deve ser concluído antes do carregamento do novo bloco. Erros sutis de temporização nos sinais `mem_valid` e `mem_ready` causavam comportamentos incorretos que só foram identificados por meio da análise detalhada das formas de onda no GTKWave. A separação clara dos módulos e a cobertura abrangente dos testes automatizados foram fundamentais para isolar e corrigir esses problemas de forma sistemática.

Uso de Inteligência Artificial

Ferramentas de IA foram utilizadas como apoio em três frentes principais: (1) **esclarecimento de conceitos** — consultas sobre o comportamento esperado da FSM em cenários de *write-back* simultâneo a *read miss*, e sobre a correta implementação dos bits *valid* e *dirty*; (2) **revisão de código** — trechos do testbench foram revisados para verificar a cobertura dos casos de borda; (3) **redação do relatório** — apoio na organização, revisão textual e formatação.

Todos os prompts seguiram o modelo *Spec-Driven Development* (SDD): cada consulta definia claramente o problema, especificava os requisitos funcionais e as restrições de interface, e indicava o comportamento esperado. Exemplos de prompts utilizados:

- “Em *SystemVerilog*, como implementar uma FSM de 3 estados para cache *direct-mapped* com *write-back*? O estado `WRITE_BACK` deve ativar `mem_rw=1` e aguardar `mem_ready` antes de transitar para `ALLOCATE`. O estado `ALLOCATE` deve ativar `mem_valid=1` (leitura) e aguardar `mem_ready` antes de retornar ao estado `IDLE` e ativar `cpu_ready`.”
- “Revisar o testbench abaixo e identificar se os cenários de *write miss* com *dirty bit* ativo estão corretamente cobertos, considerando a política *write-allocate* com *write-back*. [código inserido]”
- “Revisar o texto do relatório abaixo para clareza técnica e coesão, mantendo terminologia em inglês para os termos técnicos de arquitetura de computadores. [texto inserido]”

Toda a implementação, validação e compreensão técnica do projeto foram realizadas pelos integrantes do grupo. O uso de IA limitou-se a apoio conceitual, revisão e organização.

Referências

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface — RISC-V Edition*, 1st ed. Morgan Kaufmann, 2017.
- [2] S. Williams, *Icarus Verilog*, versão 11.0. Disponível em: <http://iverilog.icarus.com>. Acesso em: mai. 2026.

- [3] T. Baltuch and S. Williams, *GTKWave Electronic Waveform Viewer*, versão 3.3. Disponível em: <http://gtkwave.sourceforge.net>. Acesso em: mai. 2026.